ARIN5102 2025 Fall Semester Assignment #2

Yang, Fengshuo Yang

October 2025

1 Problem 1

1. **State**: A state is represented as a tuple (N, P), where:
   - $N/currentNode$ is the node where the salesman is currently at;
   - $P/passedNodes$ is a list of all nodes that have been traveled so far (including N).

   This definition aligns with the requirement 'goes through each other node in the graph once and exactly once.'
2. **Possible Initial States**: The initial state is determined by the given starting node, such as $(N_{\text{start}}, \{N_{\text{start}}\})$, where N_{start} is the starting node and $P = \{N_{\text{start}}\}$ (only the starting node is visited initially).
3. **Goal Test**: For a state (N, P): Check if P contains all nodes in the graph and $N = N_{\text{start}}$ (the salesman returns to the starting node).
4. **Operators**: In this problem, operators are possible moves between states: For a current state (N, P), the salesman will move from N to an unvisited node N', provided there is an edge between N and N' (i.e., $N' \notin P$).

   This operator generates a new state $(N', \{N'\} \cup P)$.

   Of course, when P includes all nodes, the only valid operator is moving from the current node N back to N_{start} (if an edge (N, N_{start}) exists), generating the final state $(N_{\text{start}}, \text{all\_nodes})$.
5. **Operator Costs**: The cost of moving from node N to node N' via an operator is equal to the weight (labeled cost) of the edge (N, N') in the graph. For a path consisting of consecutive moves $N_1 \to N_2 \to \cdots \to N_k$, the total cost is the sum of the edge weights: $\sum_{i=1}^{k-1} \text{weight}(N_i, N_{i+1})$.

2 Problem 2

1. **Variables**: Variables represent the word_space in the crossword grid—consecutive empty squares that must be filled with words. Each variable is uniquely identified by three attributes:

(a) Orientation: Horizontal (row-wise) or Vertical (column-wise);

(b) Starting position: Denoted as $H_{r_1,c_1,L}$ for horizontal word_space (starts at row r_1, column c_1) and $V_{c_2,r_2,L}$ for vertical word_space (starts at column c_2, row r_2);

(c) Length: L (number of squares in the word_space, equal to the length of the word filling it).

2. **Domains**: The domain of each variable (word_space) is a subset of the provided dictionary, containing only words that *match the word_space's length* (a word_space of length L cannot fit a word of other lengths). For any variable X with length L, its domain is:

$$\mathrm{D}(X) = \{w \in \text{Dictionary} \mid \text{length}(w) = L\}$$

Example: If $H_{2,3,4}$ is a horizontal word_space of length 4, $\mathrm{D}(H_{2,3,4})$ = {all 4-letter words in the dictionary} .

3. **Constraints**: Constraints are binary and ensure consistency between *intersecting word_space*: If two word_space (one horizontal, one vertical) share a square, the letter in that overlapping square must be the same for both words filling the word_space. Specifically:

(a) Let variable X (e.g., H_{r_1,c_1,L_X}) and variable Y (e.g., V_{c_2,r_2,L_Y}) intersect at the i-th position of X (1-based) and the j-th position of Y (1-based);

(b) The constraint requires: For all $w_X \in \mathrm{D}(X)$ and $w_Y \in \mathrm{D}(Y)$, the i-th character of w_X equals the j-th character of w_Y, i.e., $w_X[i] = w_Y[j]$.

Example: If $H_{2,3,4}$ (3rd character) intersects $V_{5,2,5}$ (2nd character), the constraint is $w_H[3] = w_V[2]$ for all $w_H \in \mathrm{D}(H_{2,3,4})$ and $w_V \in \mathrm{D}(V_{5,2,5})$.

3 Problem 3

3.1 Formalization as a Markov Decision Process (MDP)

A Markov Decision Process is defined by $\langle S, s_0, A, T(s,a,s'), \text{Reward}(s,a,s'), \text{End}(s), \gamma \rangle$. The components for this 3-year investment problem are specified as follows:

1. **Set of States** S
Each state is denoted $s = (y, m)$, where: - y: Current year(means at the end of which year), $y \in \{0, 1, 2, 3\}$. - m: Current money amount, $m > 0$.

2. **Starting State** s_0
Initial state is $s_0 = (0, 1)$: Year 0 with 1 unit of initial money.

3. **Terminating Condition End**(s)
$\text{End}((y, m))$ = True if and only if $y = 3$ (end of 3rd years); else False.

4. **Set of Actions** A
   $A = \{a_{\text{CD}}, a_{\text{Stock}}\}$, where: - a_{CD}: Ensure a 10% annual interest rate. - a_{Stock}: a 30% annual interest with 0.7 probability or a 10% loss for with 0.3 probability.

5. **Transition Relation** $T(s, a, s')$
   For non-terminating states $s = (y, m)$ $(y = 0, 1, 2)$:
   1. $a = a_{\text{CD}}$: Deterministic transition to $(y + 1, 1.1m)$, $T = 1$.
   2. $a = a_{\text{Stock}}$: $T((y, m), a_{\text{Stock}}, (y + 1, 0.9m)) = 0.3, T((y, m), a_{\text{Stock}}, (y + 1, 1.3m)) = 0.7$.

6. **Reward Function Reward**(s, a, s')
   Since the only key judgment is "definitely don't want to lose any money at the end of the 3 years" and the goal is to maximize the final fund:
   - Intermediate transitions $(y = 0 \to 1, 1 \to 2)$: Reward$(s, a, s') = 0$ (no intermediate rewards, as only the final result matters).
   - Terminating transitions $(y = 2 \to 3)$: Reward$(s, a, s') = m'$ (reward equals the final fund amount m', which is the target to maximize while ensuring $m' \geq 1$).

7. **Discount Factor** γ
   $\gamma = 1$: No discounting (only final reward matters, valued fully).

3.2 Computation of the Optimal Policy

Optimal policy $f(s)$ aims to maximize the expected final fund amount (with the constraint of final $m \geq 1$, i.e., no principal loss at the end of 3 years). It is computed via **backward induction**—starting from the pre-terminal year (year 2) and working backward to the initial year (year 0)—to align with the transition relation $T(s, a, s')$ and reward function Reward(s, a, s').

1. **Step 1: Year 2 ($y = 2$, pre-terminal)**
   For any state $(2, m)$ (year 2 with m units of funds), the action directly determines the final fund amount (year 3) and its expected reward:
   -a_{CD}: By $T((2, m), a_{\text{CD}}, (3, 1.1m)) = 1$, expected reward=$1.1m$.
   -a_{Stock}: By $T((2, m), a_{\text{Stock}}, (3, 1.3m)) = 0.7$ and $T((2, m), a_{\text{Stock}}, (3, 0.9m)) = 0.3$, expected reward=$m' = 0.7 \times 1.3m + 0.3 \times 0.9m = 1.18m$.

   Constraint adjustment: To ensure final $m' \geq 1$, a_{Stock} requires its minimum possible m' (0.9m) to be ≥ 1, i.e., $m \geq 1/0.9 \approx 1.111$. Thus:

$$f((2, m)) = \begin{cases} a_{\text{Stock}} & m \geq 1.111, \\ a_{\text{CD}} & m < 1.111 \end{cases}$$

2. **Step 2: Year 1 ($y = 1$, intermediate)**
   For any state $(1, m)$ (year 1 with m units of funds), the expected reward depends on the optimal action in year 2 (from Step 1) and transition relation T:

-a_{CD}: By $T((1,m),a_{\text{CD}},(2,1.1m))=1$, transitions to $(2,1.1m)$. Expected reward = optimal reward of $f((2,1.1m))$.
-a_{Stock}: By $T((1,m),a_{\text{Stock}},(2,1.3m))=0.7$ and $T((1,m),a_{\text{Stock}},(2,0.9m))=0.3$, transitions to $(2,1.3m)$ (0.7 prob) and $(2,0.9m)$ (0.3 prob). Expected reward $=0.7\times$optimal reward of $f((2,1.3m))+0.3\times$optimal reward of $f((2,0.9m))$.

Constraint adjustment: To ensure year 2's m' can satisfy the $m\geq 1.111$ or $m<1.111$ threshold (for final $m'\geq 1$), a_{Stock} requires its minimum possible m' (0.9m) to be ≥ 0.909 (so a_{CD} in year 2 gives $1.1\times 0.909\approx 1$), i.e., $m\geq 0.909/0.9\approx 1.01$. Thus:

$$f((1,m))=\begin{cases}a_{\text{Stock}} & m\geq 1.01,\\ a_{\text{CD}} & m<1.01\end{cases}$$

3. **Step 3: Year 0 ($y=0$, initial)**
Initial state is $(0,1)$ (year 0 with 1 unit of funds). Combine $T(s,a,s')$ and Step 2's optimal policy to compute expected reward:
-a_{CD}: By $T((0,1),a_{\text{CD}},(1,1.1))=1$, transitions to $(1,1.1)$. Expected reward = optimal reward of $f((1,1.1))\approx 1.5316$.
-a_{Stock}: By $T((0,1),a_{\text{Stock}},(1,1.3))=0.7$ and $T((0,1),a_{\text{Stock}},(1,0.9))=0.3$, transitions to $(1,1.3)$ (0.7 prob) and $(1,0.9)$ (0.3 prob). Expected reward $=0.7\times$optimal reward of $f((1,1.3))+0.3\times$optimal reward of $f((1,0.9))\approx 1.6430$.

Constraint check: All paths under a_{Stock} lead to final $m'\geq 1$ (e.g., $(0,1)\to(1,0.9)\to(2,0.99)\to(3,1.089)$). Thus:

$$f((0,1))=a_{\text{Stock}}$$

4. **Final Optimal Policy**
Integrating Steps 1–3, the optimal policy $f(s)$ (mapping state $s=(y,m)$ to action) is:

$$f(s)=\begin{cases}a_{\text{Stock}} & s=(0,1),\\ a_{\text{Stock}} & s=(1,m)\text{ and }m\geq 1.01,\\ a_{\text{CD}} & s=(1,m)\text{ and }m<1.01,\\ a_{\text{Stock}} & s=(2,m)\text{ and }m\geq 1.111,\\ a_{\text{CD}} & s=(2,m)\text{ and }m<1.111\end{cases}$$

4 Problem 4

4.1 Formulation as a Markov Decision Process (MDP)

1. **Set of States S**
A state $s=(pos,s_1s_2\cdots s_8,\text{loc})$ includes: - pos: Robot's current grid position (as (x,y) coordinates). - $s_1s_2\cdots s_8$: 8-bit binary sensor readings,

where $s_i = 1$ indicates the i-th adjacent cell is occupied (wall/obstacle), and 0 indicates it is free. - loc: Location flag (inside room wall: loc = in; outside obstacle: loc = out), determining the target direction (clockwise for in, counter-clockwise for out).

2. **Starting State** s_0
   Initial state $s_0 = (pos_0, s_1^0 \cdots s_8^0, \text{loc}_0)$, where: - pos_0: A position adjacent to a boundary (sensor $s_i = 1$ for at least one i). - $s_1^0 \cdots s_8^0$: Initial sensor readings with at least one $s_i = 1$ (near boundary). - loc_0: Initial location (in or out).

3. **Terminating Condition End**(s)
   End(s) = True if the robot completes a full loop around the boundary (returns to pos_0 with consistent context and sensor patterns) or permanently loses proximity (all sensors 0 for consecutive steps). Otherwise, End(s) = False.

4. **Set of Actions** A
   Four movement actions as specified: $A = \{a_{\text{north}}, a_{\text{east}}, a_{\text{south}}, a_{\text{west}}\}$, corresponding to moving up, right, down, and left, respectively.

5. **Transition Relation** $T(s, a, s')$
   For state $s = (pos, \mathbf{s}, \text{loc})$ and action $a \in A$: - Let pos' be the intended new position after action a (e.g., $pos' = (x, y-1)$ for a_{north}). - If the cell at pos' is free (indicated by the relevant sensor in $\mathbf{s}$), then $s' = (pos', \mathbf{s}', \text{loc})$, where $\mathbf{s}'$ is the updated 8-bit sensor reading for pos', and $T(s, a, s') = 1$. - If the cell at pos' is occupied (sensor indicates blocked), the robot stays at pos, so $s' = (pos, \mathbf{s}, \text{loc})$ and $T(s, a, s') = 1$ (deterministic transitions).

6. **Reward Function Reward**(s, a, s')
   Rewards enforce consistent boundary following: - $+r$: If s' maintains proximity to the boundary (at least one sensor in $\mathbf{s}'$ is 1) and movement aligns with the target direction (e.g., clockwise: a_{east} when wall is to the north for loc = in). - $-r$: If s' moves away from the boundary (all sensors in $\mathbf{s}'$ are 0) or movement contradicts the target direction (e.g., a_{west} when clockwise is required). - 0: Neutral transitions (proximity maintained but direction not clearly consistent).

7. **Discount Factor** γ
   $\gamma = 0.95$: Prioritizes immediate rewards for staying on the boundary while valuing long-term consistent looping.

4.2 Formulation as a Reinforcement Learning Problem

1. **Agent and Environment**
   - **Agent**: The boundary-following robot, equipped with 8 binary sensors (to detect occupancy of adjacent cells) and motion actuators. Its task is to learn a policy that maintains proximity to boundaries and follows

the target direction (clockwise for room interiors, counter-clockwise for obstacle exteriors).
- **Environment**: A grid-based space containing walls (room boundaries) and obstacles. It responds to the agent's actions by updating the robot's position (or keeping it stationary if movement is blocked) and generating new sensor readings, while providing reward signals based on the agent's behavior.

2. **Observation Space** O
The agent's observations include key information for decision-making: $O = (\text{pos}_{\text{rel}}, s_1 s_2 \cdots s_8, \text{loc})$, where: - pos_{rel}: The robot's position relative to the nearest boundary (e.g., "1 cell east of a wall"), simplifying spatial reasoning without relying on global coordinates. - $s_1 s_2 \cdots s_8$: 8-bit binary sensor data (same as in the MDP), where $s_i = 1$ indicates the i-th adjacent cell is occupied (wall/obstacle) and 0 indicates it is free. - loc: Location flag (in for room interiors, out for obstacle exteriors), specifying the target following direction.

3. **Set of Actions** A
Four movement actions matching grid navigation rules, with effects determined by cell availability: - a_{north}: Moves the robot one cell upward (e.g., from (x, y) to $(x, y-1)$) if the target cell is free; otherwise, the robot remains at (x, y). - a_{east}: Moves the robot one cell to the right (e.g., from (x, y) to $(x+1, y)$) if the target cell is free; otherwise, the robot remains at (x, y). - a_{south}: Moves the robot one cell downward (e.g., from (x, y) to $(x, y+1)$) if the target cell is free; otherwise, the robot remains at (x, y). - a_{west}: Moves the robot one cell to the left (e.g., from (x, y) to $(x-1, y)$) if the target cell is free; otherwise, the robot remains at (x, y).

4. **Reward Signal**
Rewards guide the agent toward consistent boundary following, matching the MDP's reward logic: - $+r$: Granted when the agent stays near the boundary (at least one $s_i = 1$ in O) and moves in the target direction (clockwise for loc = in, counter-clockwise for loc = out). - $-r$: Penalized when the agent moves away from the boundary (all $s_i = 0$) or moves opposite to the target direction (e.g., turning west when clockwise is required). - 0: Neutral reward for transitions where boundary proximity is maintained but direction alignment is unclear.

5. **Learning Objective**
The agent learns an optimal policy $\pi^*(o) = a$ (mapping observations to actions) that maximizes the expected cumulative discounted reward:

$$\max_{\pi} E\left[\sum_{t=0}^{\infty} \gamma^t r_t \mid \pi\right]$$

where r_t is the reward at time step t, and $\gamma = 0.95$ (balancing immediate and long-term rewards, consistent with the MDP).

6. **Learning Mechanism**
   - **Exploration-Exploitation**: Uses an ϵ-greedy strategy (with ϵ decreasing over time, e.g., from 0.2 to 0.01) to explore new actions while exploiting known high-reward actions.
   - **Policy Update**: For finite observation spaces, employs tabular Q-Learning to estimate $Q(o, a)$ (action-values), updated as:

$$Q(o_t, a_t) \leftarrow Q(o_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(o_{t+1}, a') - Q(o_t, a_t) \right]$$

   where $\alpha = 0.1$ is the learning rate. For larger spaces, uses function approximation (e.g., neural networks) to generalize $Q(o, a)$ across similar observations.
   - **Convergence**: Training stops when the agent maintains stable boundary following (no negative rewards for 100+ consecutive steps) or completes 3+ full boundary loops.

5 Problem 5

Using A* tree search, we define the evaluation function $f(n) = g(n) + h(n)$, where: - $g(n)$: actual cost from the initial state S to node n; - $h(n)$: heuristic estimate from node n to the goal state G.

1. **Initial State**
   - Starts with S: $g(S) = 0$, $h(S) = 7$, so $f(S) = 0 + 7 = 7$.

2. **1st Expansion (Node S)**
   - Apply operators to S:
   - O_1: $S \to A$ (cost 4). For A: $g(A) = 0 + 4 = 4$, $h(A) = 1$, so $f(A) = 4 + 1 = 5$.
   - O_2: $S \to B$ (cost 2). For B: $g(B) = 0 + 2 = 2$, $h(B) = 6$, so $f(B) = 2 + 6 = 8$.
   - Open list after expansion: $\{A\ (f = 5),\ B\ (f = 8)\}$.

3. **2nd Expansion (Node A since $f(A)$ is minimum)**
   - Apply operator O_4 to A: $A \to G$ (cost 5). For G: $g(G) = 4 + 5 = 9$, $h(G) = 0$, so $f(G) = 9 + 0 = 9$.
   - Open list after expansion: $\{B\ (f = 8),\ G\ (f = 9)\}$.

4. **3rd Expansion (Node B since $f(B)$ is minimum))**
   - Apply operator O_3 to B: $B \to A$ (cost 1). New path to A: $g(A) = 2 + 1 = 3$, $h(A) = 1$, so $f(A) = 3 + 1 = 4$.
   - Open list after expansion: $\{A\ (\text{new}, f = 4),\ G\ (f = 9)\}$.

5. **4th Expansion (Node A [new])**
   - Apply operator O_4 to A: $A \to G$ (cost 5). New path to G: $g(G) = 3 + 5 = 8$, $h(G) = 0$, so $f(G) = 8 + 0 = 8$.
   - Open list after expansion: $\{G\ (\text{new}, f = 8)\}$.

6. **Termination**
   - Select G (new, $f = 8$) from the list. The goal is reached.

The result from the expansion is as follows:

Node	**Node Order**	$g(n)$	$h(n)$	$f(n) = g(n) + h(n)$
S	1/Initial node	0	7	7
A	2/First expansion & 4/Second expansion	4	1	5
B	3/Only once expansion	2	6	8
G	5/Last expansion for the optimal path	8	0	8

The optimal solution found by A* search is: $S \xrightarrow{O_2} B \xrightarrow{O_3} A \xrightarrow{O_4} G$ with a total cost of 8.

6 Problem 6

Calculations proceed from terminal nodes upward to the root. The step-by-step process is as follows:

1. **Calculate deepest terminal nodes**
   Terminal nodes and their utilities: $M = 0$, $N = 7$, $J = 5$, $K = 7$, $L = 8$, $D = 3$, $E = 5$, $H = 4$.

2. **Calculate Node I (MIN)**
   Node I (MIN) has children M (0) and N (7). MIN selects the minimum value: $\text{Value}(I) = \min(0, 7) = 0$.

3. **Calculate Node F (MAX)**
   Node F (MAX) has children I (0) and J (5). MAX selects the maximum value: $\text{Value}(F) = \max(0, 5) = 5$.

4. **Calculate Node G (MIN)**
   Node G (MIN) has children K (7) and L (8). MIN selects the minimum value: $\text{Value}(G) = \min(7, 8) = 7$.

5. **Calculate Node C (MIN)**
   Node C (MIN) has children F (5), G (7), and H (4). MIN selects the minimum value: $\text{Value}(C) = \min(5, 7, 4) = 4$.

6. **Calculate Node B (MAX)**
   Node B (MAX) has children D (3) and E (5). MAX selects the maximum value: $\text{Value}(B) = \max(3, 5) = 5$.

7. **Calculate root Node A (MAX)**
   Node A (MAX) has children B (5) and C (4). MAX selects the maximum value: $\text{Value}(A) = \max(5, 4) = 5$.

Result Summary: Optimal value for root node A: $\boxed{5}$ and Optimal decision path: $A \to B$.

7 Problem 7

Alpha-beta pruning optimizes Minimax by cutting off subtrees that cannot affect the root's final decision.
Initial values: $\alpha = -\infty$ (MAX's best lower bound), $\beta = +\infty$ (MIN's best upper bound).

7.1 Left-to-Right Alpha-Beta Pruning

1. Process root Node A (MAX)
   Initialize $\alpha = -\infty$, $\beta = +\infty$. First visit child B (MIN).

2. Process Node B (MIN)
   -Inherit $\alpha = -\infty$, $\beta = +\infty$. Visit children in left-to-right order: $D \to E$.
   - First child D: terminal node, utility 3. MIN updates $\beta = \min(\infty, 3) = 3$.
   - Second child E: terminal node, utility 5. MIN compares 5 with current $\beta = 3$, no update.
   - Node B final value: 3. Pass back to A, which updates $\alpha = \max(-\infty, 3) = 3$.

3. Process Node C (MIN)
   -Inherit $\alpha = 3$, $\beta = +\infty$. Visit children in left-to-right order: $F \to G \to H$.
   - First child F (MAX): inherit $\alpha = 3$, $\beta = \infty$. Visit its children $I \to J$. - I (MIN): inherit $\alpha = 3$, $\beta = \infty$. Visit $M(0) \to N(7)$. I value $= \min(0, 7) = 0$. - J: terminal node, utility 5. F updates $\alpha = \max(3, \max(0, 5)) = 5$. - Node F final value: 5. Pass to C, which updates $\beta = \min(\infty, 5) = 5$.
   - Second child G (MIN): inherit $\alpha = 3$, $\beta = 5$. Visit $K(7) \to L(8)$. - First child $K = 7$: MIN updates $\beta = \min(5, 7) = 5$. - Second child $L = 8$:Since $8 > \beta = 5$,**we can prune *L***. - Node G final value: 5. Pass to C, β remains 5.
   - Third child H: terminal node, utility 4. $4 < \beta = 5$, C updates $\beta = 4$.
   - Node C final value: 4. Pass back to A, α remains 3.

4. Final Root Node A Value
   A selects $\max(3, 4) = 5$. Pruned subtrees: L (child of G).

7.2 Right-to-Left Alpha-Beta Pruning

1. Process root Node A (MAX)
   Initialize $\alpha = -\infty$, $\beta = +\infty$. First visit child B (MIN).

2. Process Node B (MIN)
   -Inherit $\alpha = -\infty$, $\beta = +\infty$. Visit children in right-to-left order: $E \to D$.
   - First child E: terminal node, utility 5. MIN updates $\beta = \min(\infty, 5) = 5$.
   - Second child D: terminal node, utility 3. MIN updates $\beta = \min(5, 3) = 3$.

- Node B final value: 3. Pass back to A, which updates $\alpha = \max(-\infty, 3) = 3$.

3. Process Node C (MIN)
   -Inherit $\alpha = 3$, $\beta = \infty$. Visit children in right-to-left order: $H \to G \to F$.
   - First child H: terminal node, utility 4. MIN updates $\beta = \min(\infty, 4) = 4$.
   - Second child G (MIN): inherit $\alpha = 3$, $\beta = 4$. Visit $L(8) \to K(7)$. - First child $L = 8$: MIN updates $\beta = \min(4, 8) = 4$. - Second child $K = 7$: $7 > \beta = 4$, **we can prune *K***. - Node G final value: 4. Pass to C, β remains 4.
   - Third child F (MAX): inherit $\alpha = 3$, $\beta = 4$. Visit $J(5) \to I$. - First child $J = 5$: $5 > \beta = 4$, **we can prune *I***. - Node F value: 5. - Node C final value: 4. Pass back to A, α remains 3.

4. Final Root Node A Value
   A selects $\max(3, 4) = 5$. Pruned subtrees: K (child of G), I (and M, N, children of I).

7.3 Result Summary

Left-to-Right Pruning: Pruned subtree is L; root value $\boxed{5}$.
Right-to-Left Pruning: Pruned subtrees are K and I (with M, N); root value $\boxed{5}$.
We can see in this problem Right-to-left traversal prunes more subtrees.